DrFresh

Ouzas Vasileios, Alino Papagiannis

April 19, 2026

1 System Overview

Our task is to implement a system that can serve liquid refreshments from a tank with the usage of two 6-12V water pumps that will be controlled through a single dual channel relay module. Our system will be based on a Raspberry Pi SBC which will be in charge of the networking, relay (and thus water pump) control and finally a simple GUI. Our system will also be able to estimate liquid usage and tank state alongside producing alerts and notifications based on current events. The GUI is specially implemented so that it can visualize tank status, usage and other statistics while also supporting the usage of command triggering. Finally, we have taken special measurements, so that we could ensure the reliability and safety of the system regarding any user errors or any errors regarding the physical and communication layer of the service.

2 Hardware Architecture

The hardware architecture of the system revolves around a Raspberry Pi acting as the main control unit. It uses a double channel relay module to control two 6-12V operating water pumps. Each pump is connected to a separate liquid tank via silicone tubing (7x10mm best ratted) and are capable of providing a maximum flow of 3L/Min. The pumps are powered with the help of a 12V external power supply, while the relay module ensures safe switching under the Pi's low-voltage GPIO signals. A DC barrel jack is used to distribute power from the external supply to the pumps.The overall setup forms a cyber-physical system where the Raspberry Pi coordinates sensing (via estimation), actuation and system control.

3 Data Flow & Communication

The Smart Drinks Dispenser follows a layered IoT communication architecture consisting of the application layer, communication layer, processing layer, and storage layer. The system uses the MQTT publish/subscribe protocol to enable lightweight and reliable message exchange between the graphical user interface (GUI) and the raspberry Pi controller.

The process begins at the application layer when the user interacts with the GUI and selects a drink option. The GUI is connected to the MQTT broker and publishes a command message to the first topic. The command message contains the selected drink and the way it will be poured(manual or automatic). The Raspberry Pi is subscribed to this topic. When the message is received, it processes the request and determines which pump must be activated, after verifying there is enough volume through the database. The Raspberry Pi then sets the corresponding GPIO pin, enabling the relay module and powering the selected pump for the requested duration.

After the dispensing cycle is completed, the Raspberry Pi generates operational feedback data. This information is published to the second topic. The GUI subscribes to this topic and receives real-time updates regarding system activity. A third data topic is also used for warning and fault notifications. Similarly, runtime errors such

as invalid requests or pump timeouts can be transmitted using the same topic with a different alert type. A fourth topic is needed for refilling. When the user refills a tank he sends a message to the Pi with the additional volume so that it can update the estimated volume in the database.

In parallel with MQTT communication, the Raspberry Pi stores each dispensing event in the local SQLite database. Stored data includes the drink type, dispensing duration, operation status, and timestamp. This enables historical tracking, volume estimation, and statistical analysis.

When historical data visualization is required, Grafana retrieves information directly from the database and presents it through dashboards. These dashboards display previous dispensing records, error conditions, and current estimated tank levels in the form of charts, tables, and gauges.

4 Data Modeling

Our system's data model follows the NGSI-LD standard. The JSON example below represents the drink dispenser as a single entity with multiple properties describing its state. Attributes such as selected drink type, pump status and dispense duration capture the system's operational behavior, while tank levels reflect the estimated remaining liquid in each container.

Listing 1: NGSI-LD Example

```
{
  "id": "urn:ngsi-ld:DrinkDispenser:001",
  "type": "DrinkDispenser",

  "drinkType": {
    "type": "Property",
    "value": "A"
  },

  "status": {
    "type": "Property",
    "value": "completed"
  },

  "dispenseDuration": {
    "type": "Property",
    "value": 5,
    "unitCode": "SEC"
  },

  "tankLevelA": {
    "type": "Property",
    "value": 65,
    "unitCode": "P1"
  },

```

```
  "tankLevelB": {
    "type": "Property",
    "value": 80,
    "unitCode": "P1"
  },

  "observedAt": {
    "type": "Property",
    "value": "2026-04-14T10:00:00Z"
  },

  "@context": [
    "https://uri.etsi.org/ngsi-ld/v1/ngsi-ld-core-context.jsonld"
  ]
}
```

The observedAt field provides temporal context for the measurements, ensuring that data updates are time-aware. Additionally, the inclusion of the standard @context enables semantic interoperability with other NGSI-LD-compliant systems and IoT platforms.

5 Data Storage Strategy

The system generates event-based data, where each dispensing action produces a timestamped record.

The main data types produced by the system are:

- **Drink Type:** identifies whether Drink A or Drink B was selected.
- **Dispensing Duration:** indicates how long the pump was activated.
- **Status:** confirms whether the dispensing operation was completed successfully or failed.
- **Timestamp:** records the exact date and time of the event.
- **Estimated Tank Level:** calculated based on cumulative usage over time.

Two storage solutions were considered: SQLite and InfluxDB.

SQLite is a lightweight relational database management system that stores all information inside a single local file. It does not require a separate database server and can run directly on the raspberry Pi with minimal system resources. Data is organized into tables consisting of rows and columns, making it easy to store structured records such as dispensing history.

InfluxDB is a specialized time-series database designed to store timestamped sensor and IoT data. Instead of traditional relational tables, it stores measurements associated with time. InfluxDB is highly optimized for high-frequency writes, efficient time-based queries, retention policies, and integration with dashboard platforms such as Grafana. This makes it a strong solution for large IoT deployments where

continuous streams of data are generated by many devices. However, InfluxDB introduces additional complexity compared to SQLite. It requires separate installation, configuration, service management, and a different query language. For a small prototype system with limited data volume, these features may be unnecessary.

Our system is expected to generate a relatively small amount of data, consisting mainly of occasional dispensing events rather than continuous high-frequency sensor streams. Therefore, SQLite provides all required functionality while keeping the implementation simple and lightweight. For these reasons we decided to go for SQLite.

6 Processing & System Logic

To begin, we designed the dispensing system to have two different serving methods depending on the user's input. If the user wants he can select to be served by default, which means that a standard volume of liquid (predefined in the Pi's code) will be dispensed. Otherwise, the user has the choice to serve himself manually, which means that by using a specific input he will start a dispensing process which will be continuous until he uses that input again to stop the dispensing process when he gets the volume of his choice (obviously there will be a limit to this custom amount). Having said that, apart from choosing dispensing method, the user is able to select of course which one of the two tanks interests him. For a typical scenario consider that John wants to use DrFresh. John would walk up to the Raspberry and view the GUI. The GUI would ask John if he want's to fill his cup with liquid from Tank1 or Tank2. John selects Tank1. The Pi will check the volume estimated in the data-base, if the estimated volume is lower than a set threshold (let's say less than 2 cups) the Pi will inform John through the GUI that the Tank that he chose doesn't have enough liquid inside and that he can choose Tank1 instead. If John chooses to get served by the tank with enough liquid inside, the Pi will wait for 3 seconds for John to place his cup under the tube, it will dispense for a defaulted period of time and then stop. After that, the Pi will publish the event to the corresponding topic.

To conclude, The system is designed around event-driven triggers and rule-based decision logic. User actions from the GUI act as primary triggers, such as selecting a tank or choosing between automatic and manual dispensing modes.

7 Reliability Considerations

The Smart Drinks Dispenser is designed to maintain safe and reliable operation under common failure conditions such as communication loss, invalid user input and hardware malfunction. If the MQTT broker or network connection becomes unavailable, the Pi can continue executing local control logic. Commands already received can still be processed, while temporary communication failures only affect remote interaction with the GUI. To prevent accidental over-dispensing, each pump operates under a maximum allowed runtime defined in software. If this limit is exceeded, the Pi automatically disables the relay. All GPIO pins are initialized in the OFF state

during startup, ensuring that pumps can't activate unintentionally after a power interruption. The database is stored locally using SQLite, reducing dependency on external servers and allowing historical data to remain available even during network outages. Finally, invalid commands received from the GUI are rejected through software validation checks, preventing unsupported operations or impossible refill values.

8 Data Visualization

The system data is visualized through a web-based dashboard that provides real-time monitoring and historical insights into the operation of the smart drink dispenser. The displayed data includes dispensing activity (number and timing of dispenses), estimated tank levels for both containers, and logged error events. These visualizations allow for long term usage analysis. The dashboard is updated at regular intervals, typically every few seconds (e.g., 2–5 seconds), ensuring near real-time feedback while avoiding excessive system load. Historical data, such as dispense events and errors, is continuously stored and can be visualized over longer time periods.

The primary end user of the system is the operator or owner of the dispenser (e.g., in an office or small business environment), rather than the casual user. The operator uses the dashboard to monitor system performance, detect issues (such as low tank levels or frequent errors), and make maintenance decisions.

Two visualization tools were considered: Grafana and ThingsBoard. Grafana is a powerful and flexible visualization platform that specializes in real-time dashboards and time-series data, making it highly suitable for monitoring IoT systems. On the other hand, ThingsBoard provides a complete IoT platform with built-in device management and dashboard capabilities, but it introduces additional complexity and overhead for smaller-scale projects. Grafana was selected as the preferred solution due to its simplicity, flexibility, and strong support for real-time data visualization.

9 Design Justification

The proposed system architecture was selected to balance simplicity, reliability and ease of implementation.

The raspberry Pi was chosen as the central controller because it combines GPIO hardware control with networking capabilities and Python software support. This allows one device to manage communication, processing, storage and actuation.

MQTT was selected as the communication protocol because it is lightweight, efficient and well suited to IoT systems based on publish/subscribe messaging.

SQLite was selected for storage because the expected data volume is small and event-driven. It avoids the complexity of running a dedicated database server while still supporting structured queries and historical analysis.

For data visualization, Grafana was chosen due to its powerful dashboard capabilities and support for real-time and historical data analysis. Its ability to present

time-series data through charts and gauges makes it particularly suitable for monitoring IoT systems.

Overall, the chosen components satisfy the project requirements while keeping implementation complexity low.

10 Implementation Reality Check

One practical challenge of the system is that tank levels are estimated rather than directly measured. Since no dedicated liquid level sensors are used, the remaining volume is calculated from historical dispensing usage. This method is cost-effective but may accumulate small errors over time.

To reduce estimation drift, the system supports refill commands through the GUI, allowing tank volumes to be manually reset whenever containers are replenished.

Another challenge is ensuring stable communication between the GUI and Raspberry Pi. Temporary network interruptions may delay commands or feedback messages. This is mitigated by storing data locally and keeping core control logic on the Raspberry Pi.

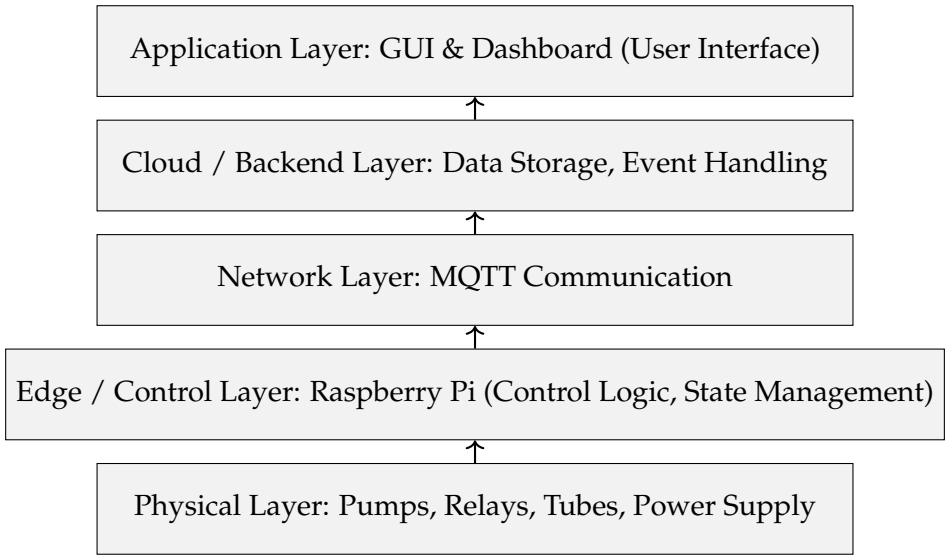

Figure 1: IoT System Architecture of the Smart Drink Dispenser